2012 年同等学力申硕计算机综合试题-软件工程

一、单项选择题 (每空 1 分，共 5 分)

1. 软件的正确性是指(B)。
A. 软件能正常运行　　B. 软件达到预期功能的程度
C. 软件符合用户的要求　　D. 软件运行中不出现任何错误
解析：考察软件质量
答案是选项 B。
软件质量的核心特性中，正确性的定义是 “软件按照预先规定的功能和性能需求，准确执行其预期任务的程度”，核心聚焦 “是否实现了预期功能”：

- 选项 A（软件能正常运行）：仅描述软件 “可运行”，未涉及功能是否符合预期（如软件能运行但计算结果错误，仍不具备正确性）；
- 选项 B（软件达到预期功能的程度）：精准匹配正确性的定义，正确性并非绝对 “无错误”，而是相对 “符合预期功能” 的程度（允许存在不影响核心功能的微小缺陷）；故选 B。
- 选项 C（软件符合用户的要求）：混淆了 “正确性” 与 “用户满意度”，用户要求可能包含非功能需求（如界面美观），而正确性仅聚焦功能是否达到预先规定的预期；
- 选项 D（软件运行中不出现任何错误）：过于绝对，软件正确性不要求 “零错误”，只需核心功能符合预期（如轻微的界面显示瑕疵不影响正确性），且完全无错误在实际软件中难以实现。

2. 结构化分析方法中的主要概念有(D)。
A. 模块内聚和耦合　　B. 对象及类
C. 模块和信息隐蔽　　D. 加工、数据流和数据源等
解析：考察软件设计-结构化分析
答案是选项 D。
结构化分析方法（Structured Analysis, SA）是面向过程的软件需求分析方法，其核心建模工具是**数据流图（DFD）**，强调从数据流动的角度理解系统功能。
DFD 的基本构成要素包括：

- 加工（Process / Transformation）：对输入数据进行处理并产生输出；
- 数据流（Data Flow）：数据在系统内部或与外部之间的移动；
- 数据存储（Data Store）：系统中保存数据的地方；
- 外部实体（External Entity / 数据源或数据潭）：系统边界外的数据提供者或接收者，也称为数据源/数据潭（Source/Sink）。

因此，选项 D 中的“加工、数据流和数据源等” 正是结构化分析方法的核心概念。
A. 模块内聚和耦合：属于结构化设计（而非分析）阶段的概念，用于模块划分；
B. 对象及类：属于面向对象方法，与结构化分析无关；
C. 模块和信息隐蔽：是结构化设计或模块化编程的思想，出现在设计阶段，非分析阶段内容。

淘宝店：PandaCode毕设文档源码库 原创

3. 软件需求很明确时，适用的软件开发模型是(C)。
A.喷泉模型　　B.增量模型　　C.瀑布模型　　D.螺旋模型
解析：考察软件过程
答案是选项 C。
不同软件开发模型的适用场景核心取决于需求的明确程度、项目风险等因素：

- 选项 C（**瀑布模型**）：是线性顺序模型，核心特征是 “阶段化、文档驱动、不可逆”，要求需求在项目初期完全明确、稳定，后续按 “需求分析→设计→编码→测试→维护” 的固定流程推进，无迭代或很少迭代，完美匹配 “需求很明确” 的场景；故选 C。
- 选项 A（**喷泉模型**）：是面向对象的迭代模型，核心是 “迭代、无明显阶段划分”，适用于需求动态变化、需持续迭代优化的项目；
- 选项 B（**增量模型**）：将软件拆分为多个可独立交付的增量，按优先级逐步开发，适用于需求基本明确但需快速交付核心功能、后续逐步完善的项目（并非 “完全明确” 的场景）；
- 选项 D（**螺旋模型**）：融合了瀑布模型和快速原型模型，核心是 “风险驱动”，通过多轮 “规划→风险分析→开发→评审” 的螺旋迭代降低风险，适用于需求不明确、风险较高的大型复杂项目。

4. 软件测试过程模型给出了(B)。
A.用例设计过程　　B.软件测试的要素以及它们之间的关系
C.白盒测试技术的要点　D.测试覆盖准则
解析：考察软件测试
答案是选项 B。
软件测试过程模型（如 V 模型、W 模型、H 模型等）的核心目的是描述软件测试活动在整个软件开发生命周期中的位置、流程及其与其他开发活动的关系。这类模型通常包括以下要素：

- 测试级别（单元测试、集成测试、系统测试、验收测试）；
- 对应的开发阶段；
- 测试目标与任务；
- 各测试活动之间的依赖和交互关系。

因此，测试过程模型不是专注于某一种测试技术（如白盒测试），也不是仅讲用例设计或覆盖准则，而是从宏观角度刻画测试作为一个系统工程所包含的要素及其相互关系。
选项分析：

- A. 用例设计过程：属于测试设计技术细节，非过程模型关注重点；
- B. 软件测试的要素以及它们之间的关系：正确，符合测试过程模型的定义；
- C. 白盒测试技术的要点：属于测试方法，与过程模型无关；
- D. 测试覆盖准则：属于测试充分性度量标准，是测试设计层面内容。

5. 教师是学校的一个部分，学校和教师之间是一种(A)关系。
A.聚合　　B.分类　　C.泛化　　D.依赖
解析：考察软件设计-面向对象方法
答案是选项 A。

淘宝店：HandaCode辉文档源码库　原创

题干核心是辨析面向对象中 “整体与部分” 的关联关系，关键在于区分不同 UML 关系的定义：

- 选项 A（聚合）：表示 “整体与部分” 的松散关联，部分可脱离整体独立存在。学校作为 “整体”，教师作为 “部分”，教师可脱离某所学校（如离职后加入其他学校）仍具有独立存在意义，完全符合聚合关系的 “可分离” 特征；故选 A。
- 选项 B（分类）：并非 UML 标准关系类型，面向对象中无此正式关联定义，属于干扰项；
- 选项 C（泛化）：表示 “一般与特殊” 的继承关系（如 “教师” 与 “大学教师”），核心是子类继承父类属性和行为，与 “整体 - 部分” 无关；
- 选项 D（依赖）：表示对象间的临时关联（如 “教师使用教案”），依赖关系中不存在 “整体与部分” 的逻辑，仅体现一方对另一方的使用依赖。

二、判断题 (每小题 1 分，共 5 分。如果正确，用 “√” 表示；否则，用 “×”表示)

1. 在对象的整个生命周期中，不同的对象可以具有相同的标识。（×）

解析：考察软件设计-面向对象方法

面向对象方法学中，**对象标识（Object Identifier, OID）是对象的唯一标识**，核心特征是 “全局唯一性”：

- 对象标识用于区分不同对象，无论对象的属性、状态如何变化，其标识在整个生命周期中保持不变；
- 不同对象（即使是同一类的不同实例）必须具有不同的标识，否则无法在系统中区分对象的独立存在（例如：两个 “教师” 对象，即使姓名、年龄相同，也需通过唯一标识区分）；
- 题干中 “不同的对象可以具有相同的标识” 违背了对象标识的唯一性原则，因此判断为错误。

2. 模块间通过参数传递基本类型的数据，模块间的这种耦合称为数据耦合。（√）

解析：考察软件设计

耦合性是模块间依赖关系的度量，**数据耦合**是软件工程中低耦合的典型类型，其核心定义与题干描述完全一致：

- 数据耦合的定义：模块之间仅通过传递**基本类型的数据参数**（如整数、字符串、布尔值等）进行交互，模块间不共享复杂数据结构（如数组、对象）、不直接访问对方的内部状态或功能，仅依赖输入输出数据的逻辑关联；
- 题干中 “模块间通过参数传递基本类型的数据” 精准匹配数据耦合的核心特征，这种耦合方式独立性强、维护成本低，是软件设计中推荐的低耦合形式；
- 需区分数据耦合与其他耦合类型（如控制耦合：传递控制信号；公共耦合：共享公共数据区），题干描述未涉及其他耦合的特征，因此判定为正确。

淘宝店：RandCode辉文档源码库，原创

3. 以结构化分析方法建立的系统模型就是数据流图。（×）

解析：考察软件设计-结构化分析

题干的核心错误在于将结构化分析方法的系统模型等同于单一的数据流图，忽略了结构化分析模型的完整性：

结构化分析的系统模型是一个 “模型体系”，核心由三大要素构成，数据流图（DFD）仅为其中之一：

- 数据流图（DFD）：描述系统的功能逻辑和数据流转（“做什么”），是动态模型；
- 数据字典：定义数据流图中所有数据元素、数据结构、加工逻辑、数据存储等的详细说明，是静态数据描述；
- 加工说明（如结构化语言、判定表、判定树）：详细描述数据流图中每个 “加工” 的具体处理规则，补充 DFD 无法体现的细节逻辑。

数据流图仅能直观展示数据的来源、处理、存储和去向，但无法说明数据的具体定义、加工的内部逻辑，必须与数据字典、加工说明配合，才能构成完整的结构化分析系统模型。

4. CMM(能力成熟度模型)的第三级是已定义级。（√）

解析：考察软件过程

该表述是正确的。CMM（能力成熟度模型）将软件过程的成熟度分为五个等级，从低到高依次为：

- **初始级**：过程不可预测且缺乏控制。
- **可重复级**：建立了基本的项目管理过程，能重复以往的成功实践。
- **已定义级**：软件过程均已文档化、标准化，并集成为组织的标准软件过程。
- **已管理级**：对软件过程和产品质量有详细的度量，并能进行定量的控制和管理。
- **优化级**：能够基于过程的量化反馈，以及新思想和新技术，持续改进过程。

因此，第三级确实是“已定义级”。该级别的核心特征是：过程已经文档化、标准化，并能够被组织内部的所有项目一致地遵循。

5. 面向对象分析方法与结构化分析方法的区别之一是一个使用“抽象”，而另一个没有使用“抽象”。（×）

解析：考察软件设计

该判断题表述错误。

“抽象”是软件工程中所有分析与设计方法共有的基本思想，并非面向对象方法所独有。

（1）**结构化分析方法**（如使用数据流图 DFD）同样运用了抽象：

- 将系统抽象为“加工”（处理）、“数据流”、“数据存储”和“外部实体”；
- 通过逐层分解（如顶层图 → 0 层图 → 1 层图）实现**功能抽象与逐步求精**；
- 忽略具体实现细节，聚焦于“做什么”而非“怎么做”。

（2）**面向对象分析方法**则通过**类、对象、继承、封装**等机制进行抽象，强调**数据与行为的统一抽象**。

因此，两种方法都使用“抽象”，区别在于抽象的方式和视角不同：

- **结构化方法：功能/过程抽象（面向过程）；**
- **面向对象方法：对象/实体抽象（面向数据与行为）。**

淘宝店 mandacode 程文档源码库 原创

题干错误地认为“结构化方法没有使用抽象”，这是对软件工程基本概念的误解。

三、简答题 (每小题 4 分，共 12 分)

1. 给出模块耦合的定义，并举例说明 3 种模块耦合。

解析：考察软件设计

（1）模块耦合的定义

模块耦合（Coupling）是指软件系统中不同模块之间相互依赖或关联的程度。耦合度越低，模块的独立性越强，系统的可理解性、可维护性和可复用性越高。因此，**低耦合是软件设计的重要原则之一。**

（2）三种模块耦合类型及示例（按耦合度从低到高排列）

①数据耦合

- 定义：模块之间通过参数表来传递基本类型的数据项。这是一种最理想、耦合度最低的形式。
- 举例：模块 A 调用模块 B，并传递一个整数参数 score。模块 B 根据 score 计算等级并返回。
  - `result = calculateGrade(score);`
- 特点：接口简单、清晰，依赖关系最小。

②标记耦合（特征耦合）

- 定义：模块之间通过参数传递一个完整的数据结构（如记录、对象），但接收模块只使用该结构中的一部分数据。
- 举例：模块 A 将一个 Student 对象（包含学号、姓名、各科成绩等）传递给模块 B，但模块 B 只使用其中的 studentId 来生成学籍报告。
  - `report = generateReport(student); // 函数内部只用了 student.id`
- 特点：耦合度高于数据耦合。如果数据结构发生变化，即使模块 B 不使用的部分发生改变，也可能导致调用代码（模块 A）需要修改。

③控制耦合

- 定义：一个模块向另一个模块传递了用于控制其内部逻辑的信号或标志。
- 举例：模块 A 调用模块 B 时，传递一个控制标志 mode。模块 B 根据 mode 是 "SAVE" 还是 "LOAD" 来决定是执行保存数据还是加载数据的操作。
  - `fileOperation("data.txt", "SAVE");`
- 特点：耦合度较高。调用模块必须了解被调用模块的内部逻辑，这使得模块 B 的功能不单一，且修改模块 B 的逻辑可能会影响到模块 A。

模块耦合的核心是 “模块间依赖的紧密程度”，**从低到高的典型耦合等级为：数据耦合 < 控制耦合 < 标记耦合 < 公共耦合 < 内容耦合**。软件设计的核心原则是 **“低耦合、高内聚”**，因此应优先采用数据耦合，避免使用公共耦合、内容耦合等强耦合类型，以提升系统的灵活性和可维护性。

2. 给出对象的关联关系定义，并举例说明。

解析：考察软件设计

（1）对象的关联关系定义

淘宝店：PandaCode编程文档源码库，原创

在面向对象方法学中，**关联关系（Association）**是指两个或多个类（或对象）之间存在的长期、稳定的语义联系，用于描述对象之间的 “相互联系” 或 “参与关系”。关联关系是 UML 中最基础的关系类型，核心特征包括：

- **双向性：**默认情况下关联是双向的（如 “学生” 与 “课程” 相互关联），也可通过箭头指定单向关联；
- **多重性：**描述关联双方的对象实例数量对应关系（如 1 对 1、1 对多、多对多）；
- **无依赖约束：**关联的对象之间是独立存在的（区别于组合、聚合的 “整体 - 部分” 关系），仅通过语义关联产生交互。

关联关系的本质是 “对象间的协作关系”，不涉及对象的创建或消亡依赖，仅明确对象在业务场景中的互动逻辑。

（2）对象的关联关系示例

- 一对一关联（1:1）：“学生” 与 “学生证” 的关联。
- 一对多关联（1:N）：“班级” 与 “学生” 的关联。
- 多对多关联（M:N）：“学生” 与 “课程” 的关联。

3. 给出模块的高内聚、低耦合原则的具体含义。

解析：考察软件设计

“高内聚、低耦合”是软件模块化设计的核心原则，旨在提高软件的可理解性、可维护性、可复用性和可测试性。

1. 高内聚（High Cohesion）：

- 含义：指一个模块内部各个元素（如函数、语句、数据）彼此密切相关，共同完成单一、明确的功能。
- 目标：模块职责单一，功能集中。
- 优点：
  - 易于理解与修改；
  - 减少模块内部逻辑混乱；
  - 提高复用可能性。
- 示例：一个名为 UserAuthentication 的模块，只负责用户登录、密码验证和会话创建，不涉及日志记录或数据库备份，即为高内聚。
- **因此，内聚性越高，模块越“专注”。**

2. 低耦合（Low Coupling）：

- 含义：指模块之间相互依赖的程度尽可能低，即一个模块对其他模块的了解和影响最小。
- 目标：模块接口简洁，交互仅通过明确定义的接口进行。
- 优点：
  - 修改一个模块不影响其他模块；
  - 便于独立开发与测试；
  - 增强系统稳定性。
- 实现方式：使用数据耦合（传递基本参数），避免全局变量、控制标志或直接访问内部数据。
- **因此，耦合度越低，模块越“独立”。**

淘宝店：pandaCode辉文档源码库，原创

四、建模题 (共 8 分)

问题陈述：在一简化的图书销售管理系统中，

● 库房管理员负责：

(1) 输入、修改、删除入库图书信息（图书名， 图书编号， 数量， 单价，入库日期）；

(2) 打印库房图书库存清单（图书名， 图书编号，库存量， 库存金额）。

● 销售员负责：

(1) 录入图书销售信息（图书名，图书编号，销售量， 单价）；

(2) 输入、修改、删除前台图书信息（图书名，图书编号，数量， 单价）；

(3) 打印前台图书库存清单（图书名， 图书编号，库存量，库存金额）。

● 部门经理负责：

(1) 统计月图书销售金额；

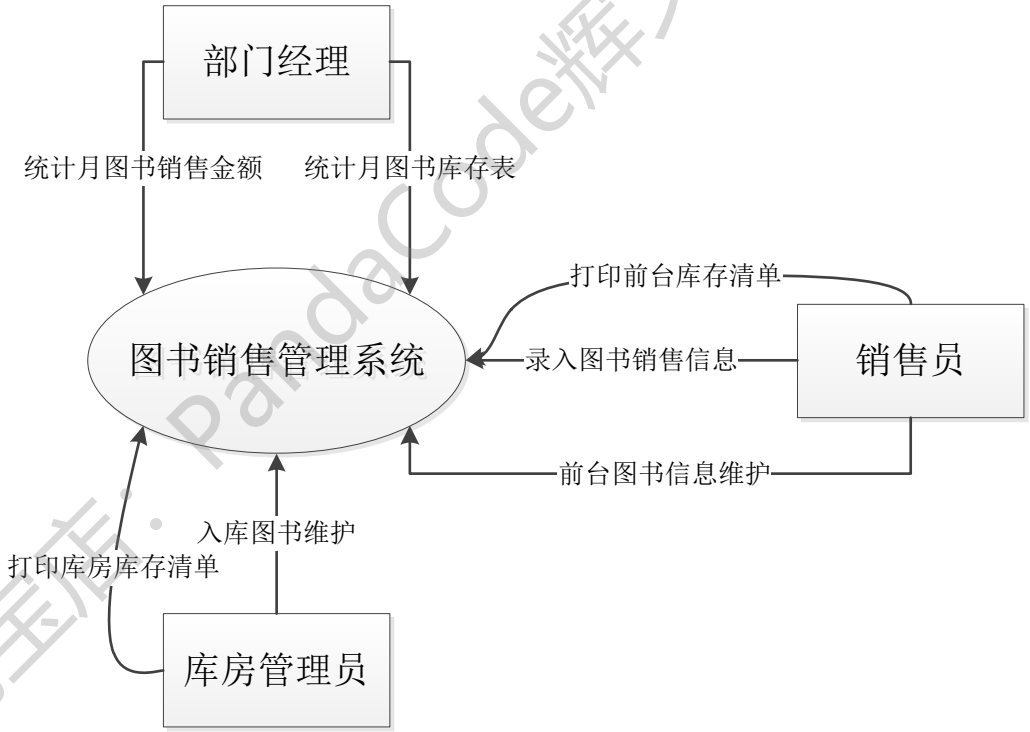
(2) 统计月图书库存情况（图书名， 图书编号， 库存量， 库存金额）。

试回答：

1. （3 分）用结构化分析方法给出该系统的顶层 DFD；
2. （2 分）给出顶层 DFD 的数据字典；
3. （3 分）选择该图书销售管理系统中的一个交互，并用顺序图来描述。

解析：考察软件设计

(1) 顶层 DFD 图（系统环境图）



(2) 顶层 DFD 数据字典：

在结构化分析方法中，**数据字典**（Data Dictionary, DD）用于对数据流图（DFD）中出现的所有**数据流、数据存储、外部实体和加工**进行精确定义。

由于是**顶层 DFD**（Context Diagram），系统被抽象为一个整体处理（通常记为 P0），因此数据字典主要包括：

淘宝店：PandaCode辉文档源码库，原创

- **外部实体（External Entities）；**
- **数据流（Data Flows）；**
- （注：顶层 DFD 中无内部数据存储和子加工，故不列数据存储项）。

顶层 DFD 数据字典如下：

- **外部实体：**
  - 库房管理员
  - 销售员
  - 部门经理
- **数据流：**
  - 入库图书信息 = 图书名 + 图书编号 + 数量 + 单价 + 入库日期
  - 图书销售信息 = 图书名 + 图书编号 + 销售量 + 单价
  - 前台图书信息 = 图书名 + 图书编号 + 数量 + 单价
  - 库房图书库存清单 ={图书名 + 图书编号 + 库存量 + 库存金额}
  - 前台图书库存清单 ={图书名 + 图书编号 + 库存量 + 库存金额}
  - 月图书销售金额报表 = 月份 + 总销售金额
  - 月图书库存情况报表 ={图书名 + 图书编号 + 库存量 + 库存金额}

(3) 选择该图书销售管理系统中的“录入图书销售信息”交互，并用次序图（顺序图）来描述：

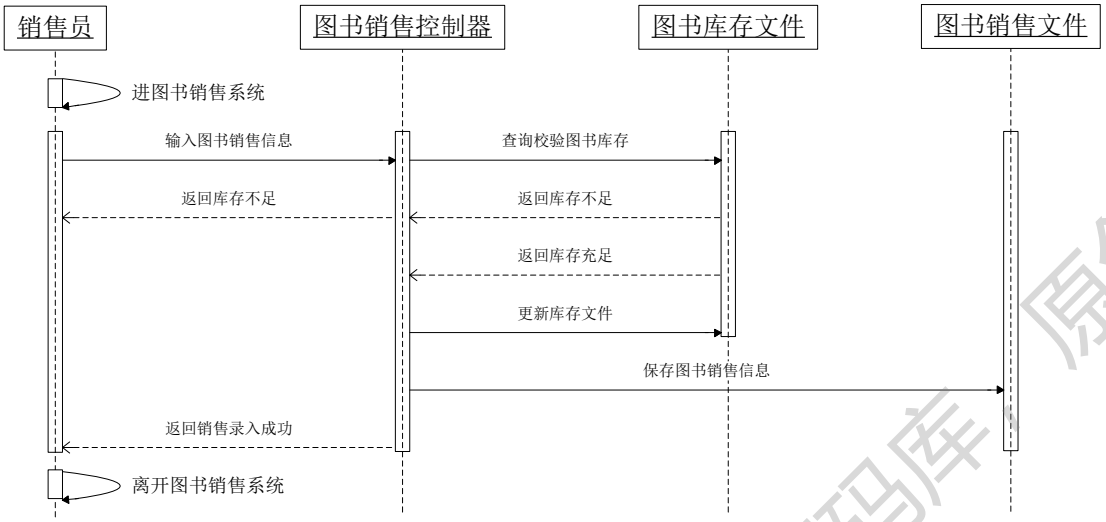
系统行为（交互）的建模工具-顺序图

定义：顺序图是一种交互图，即由一组对象以及这些对象之间的关系组成，其中还包含这些对象之间被发送的消息。

文字描述交互顺序如下：

① 销售员向图书销售系统发起录入图书销售信息请求，输入图书销售信息。
② 通过图书销售控制器，查询图书库存文件，校验库存是否充足。
③ 若库存不足，则图书销售控制器返回库存不足。
④ 销售员收到库存不足，结束本次交互。
⑤ 若库存充足，则图书销售控制器更新图书库存文件，然后保存图书销售信息到图书销售文件中。
⑥ 销售员收到销售录入成功，结束本次交互。

淘宝店：Pandaoo毕设文档源码库 原创



淘宝店：PandaCode辉文档源码库，原创